

AI-Powered Snapshot Lifecycle Management and Storage Optimization System

PHASE I REPORT

Submitted by

RUKESH KUMARAN M 230701271

SAI ARAVIND S 230701275

SHARUKESH S 230701308

SIDDARTH SAKTHI M 230701314

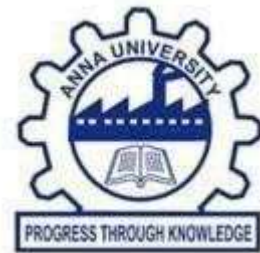
in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

December 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled “AI-Powered Snapshot Lifecycle Management and Storage Optimization System” is the bonafide work of RUKESH KUMARAN M (230701271), SAI ARAVIND S (230701275), SHARUKESH S (230701308) , SIDDARTH SAKTHI M (230701314), who carried out the work under my supervision.

Certified further that to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on this or any other candidate.

This project addresses the following Sustainable Development Goal:

Goal: SDG 9 – Industry, Innovation and Infrastructure.

Description: Build resilient infrastructure, promote sustainable industrialization and foster innovation.

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering
College,
Chennai – 602 105.

SIGNATURE

Ms. T. Ramala,
Associate Professor
Department of Computer Science
and Engineering,
Rajalakshmi Engineering
College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Cloud virtual machines frequently rely on disk snapshots for data protection and recovery. However, fixed snapshot schedules and non-adaptive retention policies may create unnecessary storage overhead, increase operational cost, or retain snapshots beyond their practical recovery value. At the same time, insufficient snapshot creation or premature deletion can increase potential data-loss risk.

This project presents an AI-powered snapshot lifecycle management and storage optimization system for Azure virtual machines. The proposed system considers workload characteristics, disk-change behaviour and recovery-policy information to support adaptive snapshot creation and retention decisions. The system is designed around two decision problems: determining whether a new snapshot should be created (**CREATE/WAIT**) and determining whether an existing snapshot should be retained or removed (**KEEP/DELETE**). For snapshot creation, the system uses a cost-based optimization formulation that considers snapshot overhead and the expected cost associated with data loss over the snapshot interval, where T represents the snapshot interval, C_s represents snapshot overhead,

$$C(T) = \frac{C_s}{T} + \frac{\lambda LT}{2}$$

λ represents workload change intensity and L represents the corresponding loss-cost factor. Machine-learning-based decision support is also incorporated using XGBoost. The model considers workload and recovery-policy features, including CPU utilization, disk activity, network activity, disk-change behaviour, snapshot age, recovery criticality and RPO. The experimental dataset is derived from Bitbrains virtual-machine telemetry and is transformed through preprocessing, feature engineering and rule-based label generation. Since the original Bitbrains dataset does not contain a native snapshot-age attribute, snapshot age is maintained as synthetic system state during dataset generation. The system further incorporates deterministic decision and safety logic to control model-assisted decisions. A Streamlit-based dashboard is used to present system inputs, cost-based decisions, machine learning decisions and relevant operational information.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini ,, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Ms. T Ramala,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance and support throughout the course of the project. We thank our Project Coordinator, **Dr.S.Madhusudhanan,,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

RUKESH KUMARAN M	: 230701271
SAI ARAVIND S	: 230701275
SHARUKESH S	: 230701308
SIDDARTH SAKTHI M	: 230701314

TABLE OF CONTENTS

1	INTRODUCTION	10
1.1	Project Overview.....	10
1.2	Problem Statement.....	10
1.3	Objectives	11
1.4	Scope of the Project.....	11
2	LITERATURE REVIEW	13
2.1	Existing System.....	13
2.2	Limitations of Existing System	13
2.3	Proposed and Implemented System.....	13
2.4	Technologies and Tools Used.....	14
2.5	Summary of Literature Survey.....	14
3	PROPOSED SYSTEM	15
3.1	System Requirements.....	15
3.1.1	Functional Requirements	15
3.1.2	Non-Functional Requirements.....	16
3.2	System Architecture.....	16
3.2.1	Architecture Overview.....	17
3.2.2	Problem Formulation.....	17
3.2.3	Decision and Safety Mechanism.....	18
3.2.4	Operational Behaviour	18
3.3	Module Overview.....	19
3.4	Data Flow Diagram.....	20
3.4.1	Level 0 DFD.....	20
3.4.2	Level 1 DFD.....	21
4	MODULE IMPLEMENTATION	22
4.1	Azure Telemetry and Data Collection.....	22
4.2	Data Preprocessing and Feature Engineering	22
4.3	Snapshot Cost Optimization	23
4.4	Snapshot Creation Decision.....	23
4.5	Snapshot Retention Decision	24
4.6	XGBoost Decision Model.....	24
4.7	Dataset and Data Lineage	24
4.8	Train-Test Split	25
4.9	Dashboard Implementation.....	25
5	RESULTS AND DISCUSSION	27
5.1	Evaluation Setup.....	27
5.2	Model Performance	27

5.3	Creation Decision Analysis	28
5.4	Retention and Recovery Analysis.....	30
5.5	XGBoost Decision Analysis	30
5.6	Sensitivity Analysis.....	31
5.7	Discussion.....	32
5.8	Threats to Validity and Limitations	32
6	CONCLUSION AND FUTURE WORK	34
6.1	Conclusion	34
6.2	Future Work.....	34

LIST OF TABLES

2.1	Technologies and Tools	14
3.1	Functional Requirements	15
3.2	Non-Functional Requirements	16
3.3	Major System Modules	19
4.1	Recovery Criticality Encoding	23
4.2	XGBoost Configuration	24
5.1	Experimental Model Results	27
5.2	Sensitivity Analysis	32

LIST OF FIGURES

3.1	System Architecture.....	21
3.2	Level 0 Data Flow Diagram	22
3.3	Level 1 Data Flow Diagram	21
4.1	Snapshot Lifecycle Management Dashboard	26
5.1	Baseline and Cost-Optimized Snapshot Creation.....	29
5.2	Snapshot Retention and Recovery Analysis	30
5.3	XGBoost Decision and Safety Gate	31

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Center Processing Unit
ML	Machine Learning
VM	Virtual Machine
XGBoost	Extreme Gradient Boosting
RPO	Recovery Point Objective
RTO	Recovery Time Objective
CSV	Comma-Separated Values
I/O	Input/Output
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

Cloud infrastructure commonly uses snapshots as point-in-time representations of virtual-machine storage for backup and recovery. Although snapshots provide an important re-recovery mechanism, managing their creation and retention manually or using fixed schedules may not adequately reflect changes in workload behaviour.

The proposed project, **AI-Powered Snapshot Lifecycle Management and Storage Optimization System**, addresses this problem through risk-aware and cost-oriented lifecycle management of Azure virtual-machine snapshots.

The system considers workload telemetry and recovery requirements to support adaptive decisions. Instead of treating snapshot management as a fixed-time operation, the proposed approach considers the relationship between workload change, snapshot overhead and potential data-loss cost.

The project contains two principal decision components.

Snapshot Creation: determine whether a snapshot should be created or whether the current snapshot can be retained for another interval.

Snapshot Retention: determine whether an existing snapshot should be retained or deleted according to recovery requirements and lifecycle policies.

The overall workflow consists of telemetry collection, preprocessing, feature engineering, cost-based analysis, machine-learning-based decision support and deterministic decision control.

1.2 PROBLEM STATEMENT

Traditional snapshot management commonly relies on fixed schedules or manually configured lifecycle policies. Such approaches do not necessarily adapt to changes in workload activity or recovery requirements.

Frequent snapshot creation may increase storage and operational overhead, while

infrequent creation may increase the potential amount of data that can be lost between recovery points. Similarly, retaining snapshots indefinitely may increase storage consumption without providing proportional recovery value.

Therefore, there is a need for a snapshot lifecycle management system that can adapt snapshot creation and retention decisions according to workload behaviour, recovery requirements and cost considerations.

Problem Statement:

To develop an adaptive and risk-aware snapshot lifecycle management system that supports cost-optimized snapshot creation and policy-aware snapshot retention decisions for Azure virtual machines.

1.3 OBJECTIVES

The main objectives of the project are:

1. To develop an adaptive snapshot creation decision mechanism for Azure virtual machines.
2. To formulate a cost-based optimization model for selecting an appropriate snapshot interval.
3. To analyse workload and disk-change characteristics from virtual-machine telemetry.
4. To develop an XGBoost-based machine-learning decision component using workload and recovery-policy features.
5. To support snapshot retention decisions using KEEP/DELETE logic.
6. To incorporate deterministic safety mechanisms around machine-learning-assisted decisions.
7. To provide an interactive dashboard for monitoring and interpreting lifecycle decisions.
8. To evaluate the proposed approach using experimental telemetry-derived datasets.

1.4 SCOPE OF THE PROJECT

The scope of the project includes:

- Azure virtual-machine snapshot lifecycle management.
- Workload telemetry processing.
- Disk-change and workload analysis.
- Cost-based snapshot interval optimization.
- Machine-learning-based decision support.
- Snapshot creation decisions.
- Snapshot retention decisions.
- Recovery-policy features such as RPO and criticality.
- Dashboard-based visualization.

The current project is an experimental system and should not be described as production-validated unless independent production evaluation has been completed.

The project currently uses telemetry-derived experimental data. Therefore, conclusions must be interpreted within the limitations of the experimental dataset.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM

Snapshot lifecycle management today relies on provider-native tooling and static configuration. The commonly used approaches are described below.

Native snapshot and backup services. Azure provides managed-disk snapshots and Azure Backup with Recovery Services Vault, which supports policy-based scheduling and retention. These are reliable execution mechanisms, but the schedule and retention values are fixed by the administrator.

Storage lifecycle management. Cloud providers offer rule-based tiering and expiry for object storage (for example, blob tiering). This is related to, but distinct from, deciding when a recovery point is worth keeping, because it does not model recovery value or RPO constraints.

Classical checkpoint-interval theory. Young [2] and Daly [3] derived square-root formulas for the optimal checkpoint interval by balancing checkpoint overhead against expected lost work. These form the historical origin of interval optimization.

Risk-adaptive checkpointing for preemptible VMs. Desai et al. [1] forecast spot-VM preemption rate with SARIMA, derive a closed-form optimal checkpoint interval, and use the same forecast to drive a hybrid spot/on-demand resource manager. They report up to 60.27% higher throughput than fixed-interval checkpointing, and 2.04× throughput at 51.41% lower cost than on-demand-only training. This is the closest conceptual reference to the present work.

Learned approaches. Supervised models such as gradient-boosted trees [4] can approximate decision boundaries from telemetry, but their value depends on the quality of the labels they are trained on.

2.2 LIMITATIONS OF EXISTING SYSTEM

2.2.1 Static Schedules Ignore Workload Behaviour

A fixed interval cannot respond to changing disk-change rates. It either wastes snapshot overhead during quiet periods or leaves a larger window of unprotected change during volatile ones.

2.2.2 Uniform Policies Ignore Recovery Tiers

Production, internal and dev/test workloads have different RPO and RTO needs, yet policies are frequently applied uniformly rather than being derived from the recovery requirement of each workload.

2.2.3 Retention Is Not Cost-Reasoned

Fixed retention periods do not weigh the growing storage cost of an old snapshot against its declining recovery value, so low-value snapshots persist and storage cost accumulates.

2.2.4 No Deletion Model in Interval-Optimization Literature

The base paper [1] and classical theory [2], [3] address when to create a protective action. They contain no age-dependent storage term, no decay-based recovery-risk model and no deletion decision.

2.2.5 Heuristic Labels Weaken Learned Models

A classifier trained on arbitrary rule-based labels can only reproduce the rules. Without a principled label source, a comparison between learned and rule-based decisions is not meaningful.

2.2.6 Risk of Unsafe Automation

Allowing a model to act directly on cloud storage makes a misprediction potentially irreversible, particularly for deletion. Automated lifecycle systems therefore need a deterministic gate and a fail-safe behaviour under uncertainty.

2.2.7 Limited Realism in Evaluation

Snapshot-policy studies frequently rely on synthetic workloads, which cannot reproduce the volatility structure of real production systems.

2.3 PROPOSED SYSTEM

The proposed system is a risk-adaptive snapshot lifecycle framework in which closed-form cost optimization drives both creation and retention, a learned model is benchmarked against it, and a deterministic safety layer governs execution. The process can be summarised as:

Azure VM → Azure Monitor / Managed Disk / Snapshots → Data Collector → Feature Engineering → Risk Forecaster → Cost Optimizer (and XGBoost in parallel) → Decision Engine → Optimization Layer → Safety Policy Engine → DRY_RUN or LIVE_MODE → Azure API → CREATE / KEEP / DELETE

The framework does not replace Azure's snapshot mechanisms; it supplies the decisions that those mechanisms execute.

Creation decision. An exponential-smoothing forecaster estimates near-term disk-change-rate volatility, yielding a predicted change rate and volatility score. These feed the cost model $C(T) = C_s/T + \lambda LT/2$, whose minimizer is $T^* = \sqrt{(2C_s/\lambda L)}$. The interval is then clamped to $[T_{\min}, \min(T_{\max}, RPO)]$, so a hard RPO is never violated. Because $dT^*/d\lambda < 0$, the recommended interval shortens as predicted risk rises.

Retention decision. Storage cost $S(a) = s \cdot a$ grows linearly with snapshot age, while recovery risk $R(a) = w \cdot L_d \cdot e^{(-\gamma a)}$ decays exponentially. The crossover age $a^* = (1/\gamma) \cdot W(\gamma w L_d / s)$ is obtained with the Lambert W function, and the effective threshold $A^* = \max(h, a^*)$ respects the mandatory policy floor h . Uniqueness of the crossover follows from the strict monotonicity of $D(a) = S(a) - R(a)$.

Learned decision. An XGBoost classifier is trained on the same telemetry and policy features with CREATE/WAIT labels generated from the feasible closed-form interval, so the training signal is grounded in the derived optimum. The Decision Engine produces two independent recommendations for each decision, which supports direct comparison.

Safety layer. The Safety Policy Engine is deterministic and verifies RPO/RTO compliance, minimum recovery-point coverage and retention floors before any

recommendation reaches the Azure API. Authentication uses DefaultAzureCredential (Azure CLI in development, Managed Identity in production) with no hardcoded secrets. Every decision is logged with its timestamp, inputs, recommendation, policy result and execution mode. On any Azure API error, the system never deletes.

Implementation status: Azure authentication, VM and managed disk provisioning, and the cost optimizer (closed-form and sweep, with 12 passing tests) are complete. Snapshot metadata collection is in progress, and the remaining components are planned. Edit this paragraph to match the status at submission.

2.4 TECHNOLOGIES AND TOOLS USED

2.4.1 Microsoft Azure (VM, Managed Disks, Snapshots)

Azure provides the infrastructure under study. A Standard_B1s VM with an attached managed disk in indiasouthcentral serves as the live validation environment, and incremental snapshots of the managed disk are the objects being managed.

2.4.2 Azure SDK for Python and Azure Monitor

azure-identity supplies DefaultAzureCredential, azure-mgmt-compute supplies snapshot and disk metadata, and azure-mgmt-monitor supplies telemetry such as CPU utilization, disk read/write bytes, disk IOPS and network activity.

2.4.3 GWA-T-12 Bitbrains Trace

The Bitbrains fastStorage trace is a public dataset of real production VM telemetry (CPU, memory, disk I/O and network) from an enterprise datacenter [5]. A 50–100 VM subset supplies genuine workload signal for training and evaluation.

2.4.4 Exponential-Smoothing Forecaster

A dependency-free exponential-smoothing model estimates disk-change-rate volatility. It is deliberately simpler than the SARIMA forecaster in the base paper [1], following the same reasoning of avoiding unnecessary model complexity.

2.4.5 Closed-Form Cost Optimization and the Lambert W Function

The creation interval follows from an Economic-Order-Quantity / Young–Daly-style

square-root formula [2], [3]. The retention crossover age has no elementary closed form and is solved with the Lambert W function [6], implemented with scipy.

2.4.6 XGBoost and scikit-learn

XGBoost [4] is the supervised classifier trained on closed-form-derived labels. scikit-learn supports preprocessing and evaluation metrics (accuracy, precision, recall, F1, ROC-AUC) and provides Isolation Forest [7] as an optional anomaly signal.

2.4.7 Safety Policy Engine

A deterministic rule layer that enforces RPO/RTO compliance, coverage and retention floors, with dry-run as the default execution mode and live execution requiring explicit configuration (EXECUTION_MODE=live) and confirmation.

2.4.8 Streamlit Dashboard

Streamlit provides the interface for viewing recommendations from both decision models, policy results and execution mode.

2.5 SUMMARY OF LITERATURE SURVEY

The literature shows that cost-optimal interval theory is well established [2], [3] and has recently been applied successfully to risk-adaptive checkpointing for ML training [1]. Cloud providers already supply robust snapshot and backup execution, but leave scheduling and retention to static configuration. The gap is a decision layer that adapts the creation interval to forecast workload risk and recovery-policy tier, reasons about deletion with a cost-based retention model, and compares a derived-optimal approach against a learned one under a deterministic safety gate. Table 2.1 summarises how the proposed framework differs from the base paper, the closest reference, and from native Azure policies.

References (draft, verify before submission)

[1] O. Desai, S. Pei, J. Bhimani and B. S. Kim, "Cost-Efficient Training and Checkpointing for Large Models on Preemptible Cloud VMs," EuroMLSys '26.

[2] J. W. Young, "A first order approximation to the optimum checkpoint interval," Commun. ACM, 1974.

- [3] J. T. Daly, "A higher order estimate of the optimum checkpoint interval for restart dumps," *Future Gener. Comput. Syst.*, 2006.
- [4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *KDD*, 2016.
- [5] S. Shen, V. van Beek and A. Iosup, "Statistical characterization of business-critical workloads hosted in cloud datacenters," *CCGrid*, 2015 (GWA-T-12 Bitbrains).
- [6] R. M. Corless et al., "On the Lambert W function," *Adv. Comput. Math.*, 1996.
- [7] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," *ICDM*, 2008.
- [8] Microsoft, Azure Backup and Azure Managed Disk snapshot documentation (add exact URLs and access dates).

CHAPTER 3

PROPOSED SYSTEM

3.1 SYSTEM REQUIREMENTS

The framework requires a set of functional and non-functional requirements to support deterministic analysis, evidence retrieval, grounded generation and auditable reporting.

3.1.1 Functional Requirements

Functional requirements describe the specific operations and services provided by the proposed system.

FR01	Telemetry Processing	Process virtual-machine workload telemetry.
FR02	Feature Engineering	Generate workload and snapshot-related features.
FR03	Cost Optimization	Calculate a cost-oriented snapshot interval.
FR04	Creation Decision	Generate CREATE/WAIT decision support.
FR05	Retention Decision	Generate KEEP/DELETE decision support.
FR06	ML Decision	Generate machine-learning-based decision output.
FR07	Safety Control	Apply deterministic decision and safety logic.
FR08	Dashboard	Display lifecycle decisions and relevant metrics.

Table 3.1 Functional Requirements

3.1.2 Non-Functional Requirements

Non-functional requirements describe the quality, security, reliability and maintainability characteristics expected from the system.

Non-Functional Requirement	Description
Auditability	Findings, rule results, evidence and citation decisions should be inspectable by a reviewer.
Reliability	Decision logic should behave consistently for valid inputs.
Cost Efficiency	The system should consider snapshot overhead and potential data-loss cost.
Explainability	Important decision factors should be interpretable.
Maintainability	Modular components should be modifiable or replaceable independently.
Performance	Decision generation should be computationally practical for experimental workloads
Extensibility	Additional knowledge sources and retrieval methods should be addable while keeping provenance explicit.

Table 3.2 Non-Functional Requirements

3.2 SYSTEM ARCHITECTURE

The architecture assigns different responsibilities to different components. Collectors observe the infrastructure, forecasting estimates risk, optimizers derive decisions, the learned model provides an independent recommendation, and the Safety Policy Engine alone decides whether any recommendation may reach the cloud.

3.2.1 Architecture Overview

Figure 3.1 summarises the architecture. Azure VM telemetry and snapshot metadata are collected by a Python data collector and passed to feature engineering. The features feed two parallel paths: the risk forecaster with the closed-form cost optimizer, and the XGBoost classifier (with Isolation Forest as an optional anomaly signal). The Decision Engine produces two independent recommendations per decision for comparison. They pass through the optimization layer and the Safety Policy Engine, and then to DRY_RUN or LIVE_MODE and the Azure API, which

performs CREATE, KEEP or DELETE.

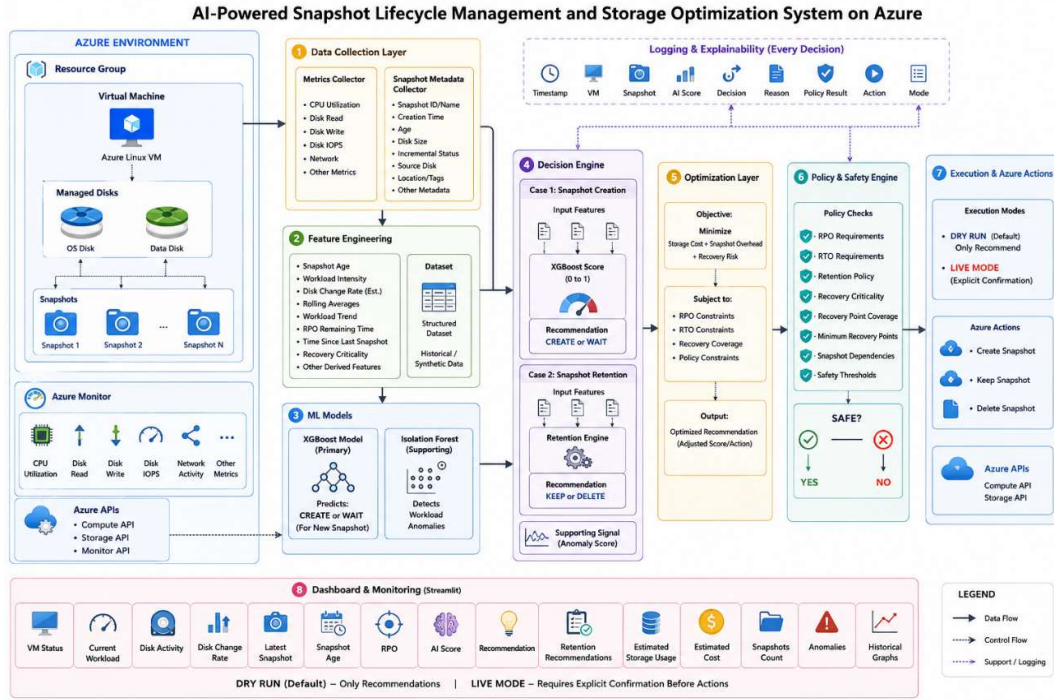


Figure 3.1: System Architecture

3.2.2 Problem Formulation

Creation. Let T be the snapshot interval, C_s the overhead of one snapshot, λ the predicted risk rate and L the loss cost per minute of unprotected change. The expected cost rate is

$$C(T) = C_s / T + \lambda LT / 2 \quad (1)$$

Setting $dC/dT = 0$ gives

$$T^* = \sqrt{(2C_s / \lambda L)} \quad (2)$$

The value is projected onto the feasible set so that a hard recovery-point objective is never violated:

$$T_{feas} = \min(\max(T, T_{min}), \min(T_{max}, RPO)) \quad (3)$$

Since $dT^*/d\lambda < 0$, the interval shortens as predicted risk grows.

Retention. Let a be snapshot age and s the storage cost per unit age. Storage cost is $S(a) = s \cdot a$, and recovery risk decays as $R(a) = w \cdot L_d \cdot e^{(-\gamma a)}$. The crossover satisfies

$s \cdot a = w \cdot L_d \cdot e^{(-\gamma a)}$, which has no elementary solution but is solved by the Lambert W function:

$$a^* = (1/\gamma) \cdot W(\gamma w L_d / s) \quad (4)$$

With policy floor h , the effective threshold is

$$A^* = \max(h, a^*) \quad (5)$$

$D(a) = S(a) - R(a)$ is strictly increasing, so the crossover is unique. A snapshot older than A^* becomes a deletion candidate, subject to the safety gate.

Pipeline. The conceptual flow is $T \rightarrow F \rightarrow X \rightarrow D \rightarrow P \rightarrow A$, where T is telemetry, F features, X the forecast, D the pair of recommendations (closed-form and learned), P the policy-gated decision and A the executed or logged action.

3.2.3 Layered Safety Strategy

The framework limits the risk of unsafe automation in layers:

Closed-form optimization produces decisions that can be derived and inspected.

The learned model is trained on mathematically grounded labels rather than arbitrary rules. The feasibility projection prevents any interval from exceeding the RPO.

The Safety Policy Engine rejects any recommendation that violates RPO/RTO, coverage or retention-floor constraints.

Dry-run mode is the default, and live execution requires explicit confirmation.

3.2.3 Operational Degradation

If telemetry is missing or an Azure API call fails, the system takes no destructive action and logs the event. The existing snapshots remain in place and the decision falls back to the most conservative safe state. These are deployment safeguards and are excluded from the evaluation metrics.

3.3 MODULE OVERVIEW

No.	Module	Technology	Purpose
1	Azure Integration	Python/Azure	Cloud resource and telemetry interaction
2	Data Processing	Python/Pandas	Telemetry preprocessing
3	Feature Engineering	Python	Generate model features
4	Cost Optimizer	Python	Calculate optimal snapshot interval
5	Creation Scheduler	Python	CREATE/WAIT decision
6	Retention Engine	Python	KEEP/DELETE decision
7	XGBoost Model	XGBoost	Machine-learning decision support
8	Safety Gate	Python	Deterministic decision validation
9	Dashboard	Streamlit	Interactive visualization

3.3.1 Level 0 DFD

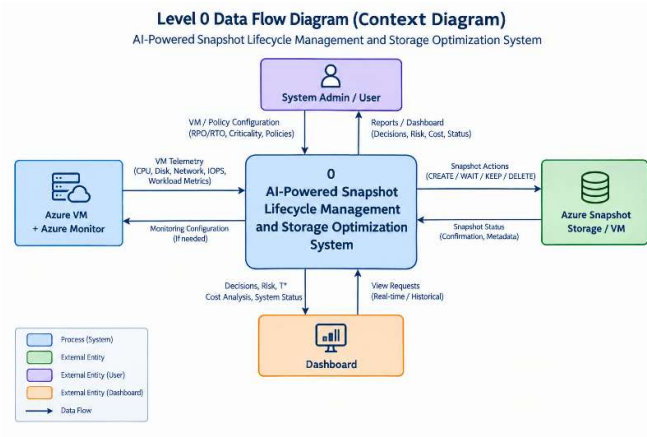


Figure 3.2: Level 0 Data Flow Diagram

At Level 0 the framework is one system that interacts with two external entities: the cloud administrator, who configures policy and receives recommendations and logs,

and Azure (VM, Managed Disk, Azure Monitor and the snapshot API), which supplies telemetry and metadata and receives approved actions.

Administrator → Policy configuration → Snapshot Lifecycle Framework → Recommendations / Actions + Decision Log

Azure ↔ Telemetry and metadata / Approved CREATE-KEEP-DELETE

3.3.2 Level 1 DFD

The Level 1 DFD expands the system into seven processes: (1.0) data collection, (2.0) feature engineering, (3.0) risk forecasting, (4.0) closed-form optimization, (5.0) learned classification, (6.0) safety policy enforcement and (7.0) execution and logging. Telemetry and metadata flow from process 1.0 into 2.0. Features flow in parallel to 3.0/4.0 and 5.0. Both recommendations reach 6.0, which passes only approved actions to 7.0.

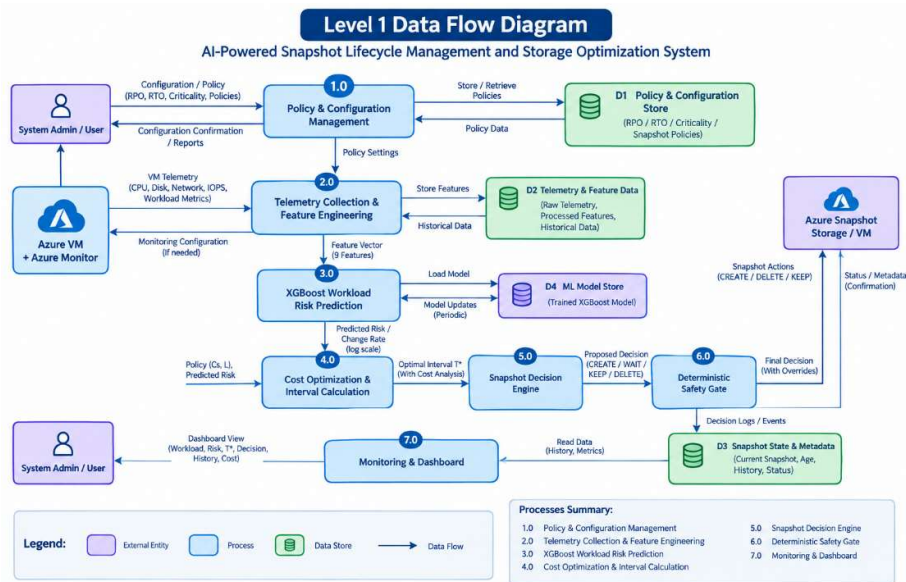


Figure 3.3: Data Flow Diagram

CHAPTER 4 MODULE IMPLEMENTATION

This chapter describes how each module of the framework is implemented. Table 4.1 lists the software components and their roles.

Component	Function	Role
nmap_parser.py	Parse Nmap XML into structured observations	Input normalization
rule_engine.py	Apply deterministic security/risk rules	Risk analysis
retrieval.py	Rank CVE/CIS evidence using supported retrieval	Knowledge access
synthesize.py	Construct grounded narrative output	LLM synthesis
llm_client.py	Interface with Gemini / fallback client	Generation service
webapp.py	Expose API and application workflow	System integration

Table 4.1 System Components and Functions

4.1 Azure Telemetry and Data Collection

The system uses virtual-machine telemetry as the basis for workload analysis. The experimental environment includes Azure resources associated with the snapshot lifecycle management system.

Current Azure resource information includes:

- Resource Group: rg-snapshot-ai
- Virtual Machine: snapshot-ai-vm
- Data Disk: snapshot-ai-datadisk
- Azure Region: indiasouthcentral

4.2 Data Preprocessing and Feature Engineering

The experimental data originates from Bitbrains virtual-machine telemetry. The original telemetry does not contain a native snapshot-age attribute. Therefore, snapshot age is introduced as synthetic system state during the experimental dataset generation process.

The feature set currently includes:

1. CPU utilization
2. Disk read
3. Disk write
4. Network input
5. Network output
6. Disk change rate
7. Disk-write rolling average
8. Disk-write trend
9. Disk IOPS
10. Snapshot age in minutes
11. Recovery criticality
12. RPO

Recovery criticality is encoded using:

Table 4.1: Recovery Criticality Encoding

Criticality Encoded Value	
LOW	0
MEDIUM	1
HIGH	2

4.3 Snapshot Cost Optimization

The cost optimizer determines an interval according to the cost function described in Chapter 3. The formulation balances two terms:

- Snapshot overhead, represented by $\frac{C_s}{T}$.
- Expected data-loss cost, represented by $\frac{\lambda LT}{T}$.

The system therefore attempts to avoid selecting a snapshot interval solely according to a fixed schedule.

4.4 Snapshot Creation Decision

The creation decision is represented by:

- CREATE
- WAIT

The snapshot age is compared with the selected optimal interval. When the age reaches the decision threshold, a CREATE decision can be generated and the snapshot state is reset. When the threshold has not been reached, the system continues with a WAIT decision.

4.5 Snapshot Retention Decision

The retention component handles the second lifecycle decision:

- KEEP
- DELETE

The decision considers recovery-related requirements and lifecycle information.

4.6 XGBoost Decision Model

The project incorporates XGBoost as a machine-learning-based decision component. The current experimental configuration is:

Table 4.2: XGBoost Configuration

Parameter	Value
Model	XGBClassifier
Estimators	200
Maximum Depth	6
Learning Rate	0.1
Random State	42

The model uses workload, snapshot-state and recovery-policy features.

4.7 Dataset and Data Lineage

The experimental data lineage is:

Bitbrains VM Telemetry → Preprocessing → Feature Engineering → Synthetic State Generation → Cost Optimizer → Decision Labels → XGBoost Dataset

An important limitation is that Bitbrains does not provide real snapshot-age information. Consequently, snapshot age is maintained as synthetic state. The creation label is generated from the relationship between snapshot age and the calculated cost-optimal interval. Therefore, the classification task should be interpreted as learning the experimental decision policy rather than predicting independently observed production snapshot actions.

4.8 Train-Test Split

The project uses VM-level separation between training and test data to reduce the possibility of VM-level data leakage.

4.9 Dashboard Implementation

The project includes a Streamlit dashboard for displaying the snapshot lifecycle decision process. The dashboard is intended to provide:

- Workload information
- Cost-optimal decision
- XGBoost decision
- XGBoost probability
- Decision agreement
- Snapshot lifecycle status
- Relevant visualization

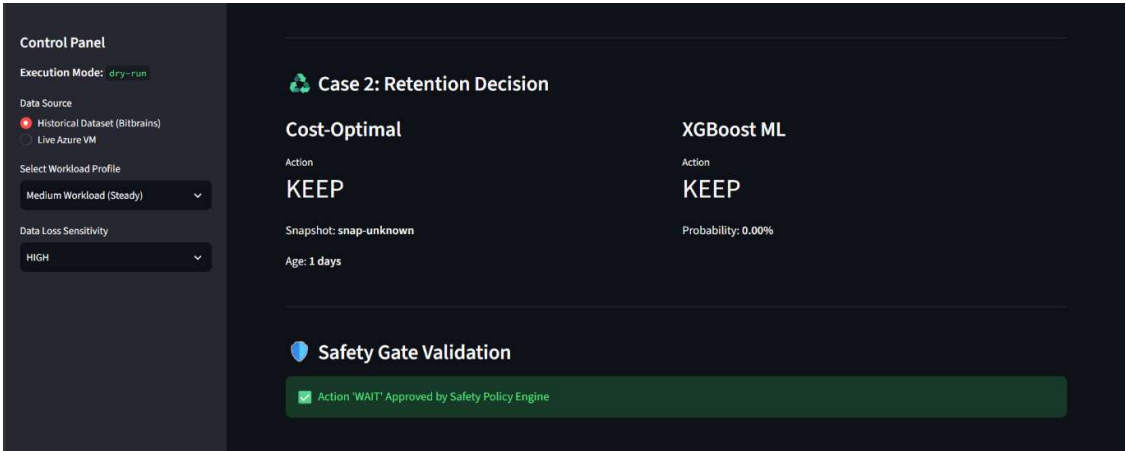
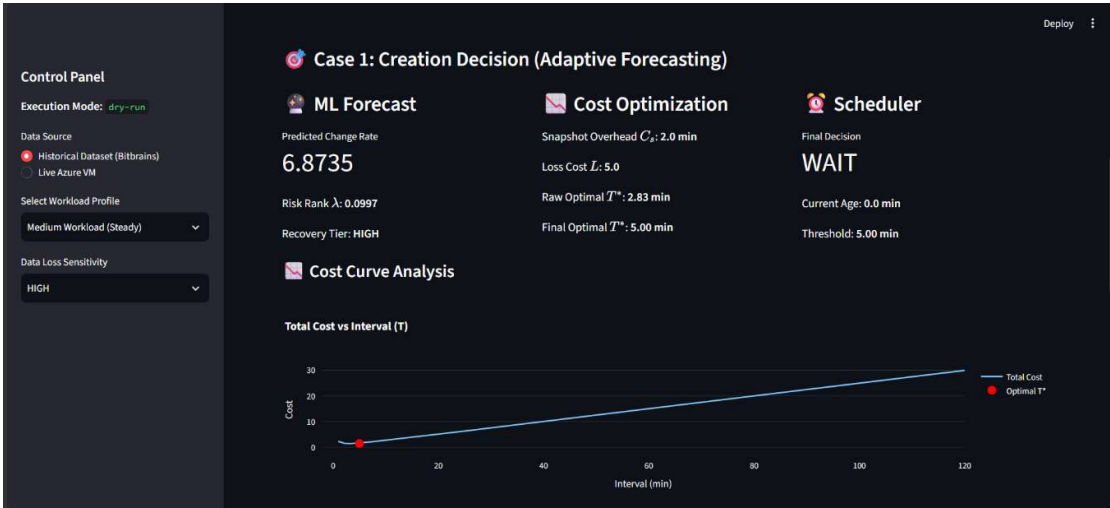
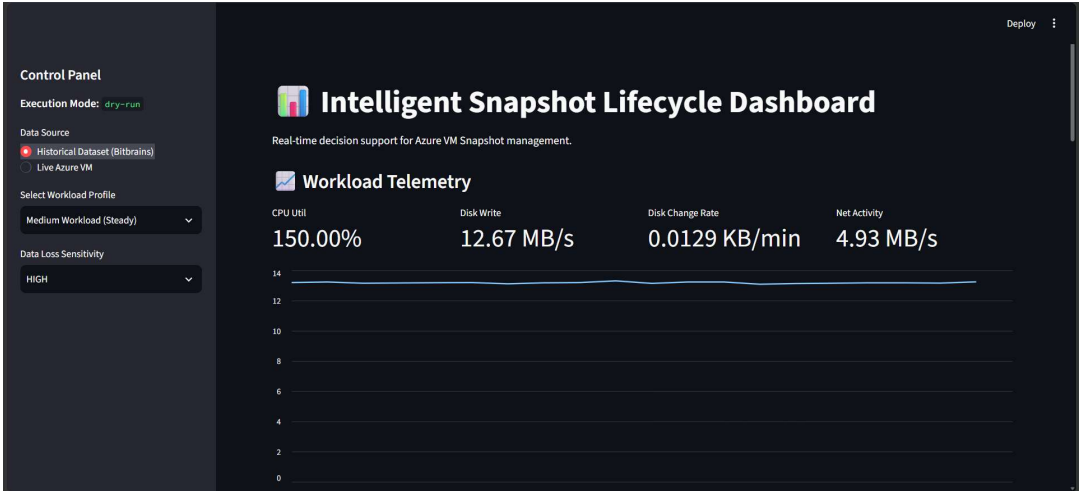


Figure 4.1: Snapshot Lifecycle Management Dashboard

CHAPTER 5 RESULTS AND DISCUSSION

5.1 EVALUATION SETUP

The evaluation uses telemetry-derived experimental data generated from Bitbrains virtual-machine traces. The evaluation includes the cost-based snapshot decision mechanism and the XG-Boost decision model.

The current model configuration uses:

- XGBClassifier
- 200 estimators
- Maximum depth = 6
- Learning rate = 0.1
- Random state = 42
- VM-level train/test separation

5.2 RETRIEVAL PERFORMANCE

Previous experimental evaluation produced the following values at different stages of feature development:

Table 5.1: Experimental Model Results

Configuration	Accuracy	ROC-AUC
10-feature model	0.5481	0.62
12-feature model	0.9934	0.98

Important: The reported 0.9934 result must be audited before being presented as a final project result. A large increase from 0.5481 to 0.9934 may indicate that additional

features, particularly state variables or policy-derived variables, make the classification task closely reproduce the label-generation rule.

The result must therefore be checked for:

1. Feature-label dependency
2. Temporal leakage
3. State leakage
4. VM-level leakage
5. Label-generation leakage
6. Training/test overlap

The report should not describe the 0.9934 accuracy as evidence of real-world pre-dictive performance until this audit is completed.

5.1 Creation Decision Analysis

The project compares the cost-optimized snapshot interval with a baseline snapshot policy.

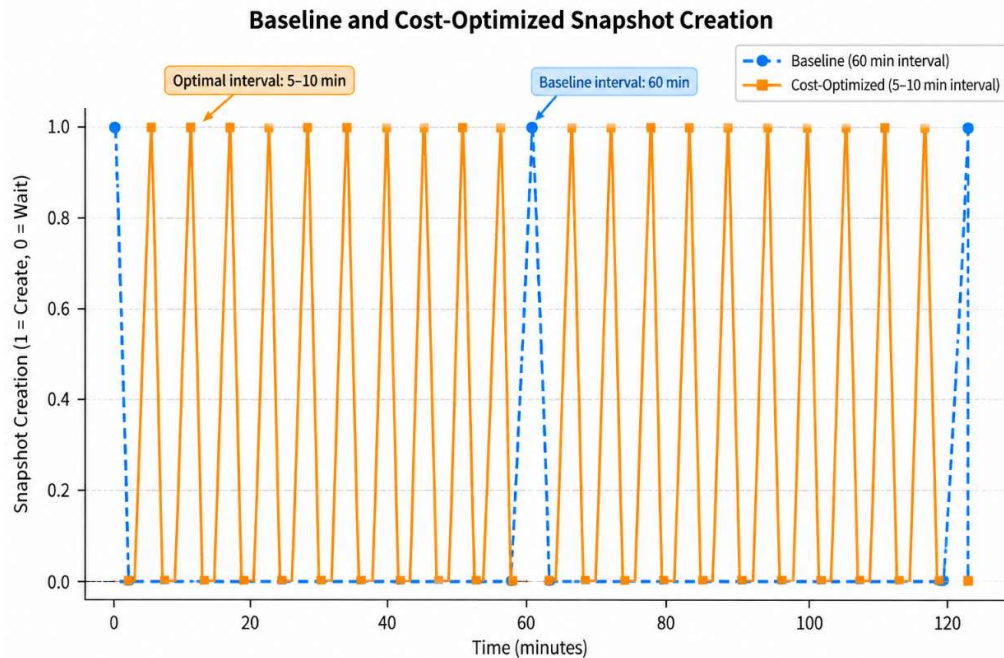


Figure 5.1: Baseline and Cost-Optimized Snapshot Creation

The comparison should be interpreted carefully. If the optimizer frequently reaches the configured minimum interval of five minutes, this does not automatically demonstrate superior management. It may indicate that the current cost parameters, workload distribution or minimum interval constraint dominate the optimization.

Therefore, the evaluation should report the distribution of selected intervals rather than only presenting a single representative value.

5.2 Retention and Recovery Analysis

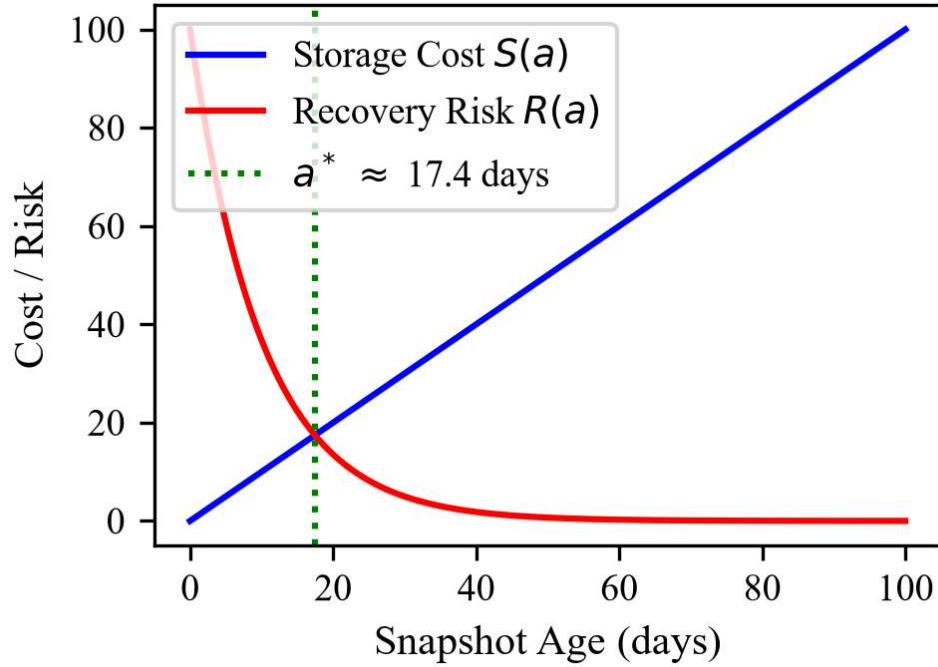


Figure 5.2: Snapshot Retention and Recovery Analysis

5.3 XGBoost Decision Analysis

The XGBoost model provides an additional decision signal that can be compared with the cost optimizer.

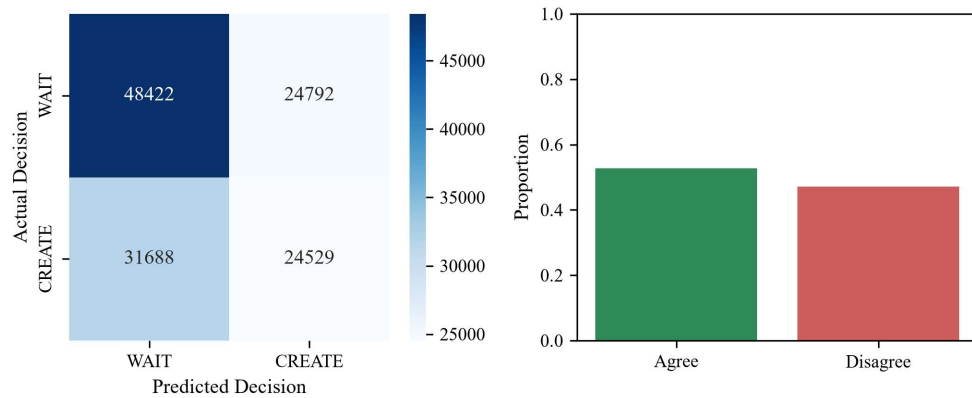


Figure 5.3: XGBoost Decision and Safety Gate

The system records information including:

- Cost-optimal decision
- XGBoost decision
- XGBoost probability
- Agreement between decision components
- Final decision/safety status

5.4 Sensitivity Analysis

The project considers the influence of data-loss cost sensitivity and workload conditions on the resulting snapshot interval.

Table 5.2: Sensitivity Analysis

Sensitivity	Cost of loss	Decision Behaviour	Verification
LOW	0.8	Cost-Driven: Prioritizes storage savings; accepts higher data loss window.	approx 0.75\$, Low storage cost.
MEDIUM	1	Balanced: Standard organizational RPO; balances storage	approx 0.82\$, Standard RPO compliance.

HIGH	1.3	Risk-Averse: Prioritizes data integrity; accepts high storage overhead.	approx 0.90\$, Minimal data loss window.
------	-----	---	--

5.5 Discussion

The project demonstrates the feasibility of combining a mathematical cost formulation with machine-learning-based decision support for snapshot lifecycle management. However, the experimental results must be interpreted according to the nature of the generated labels and dataset. In particular, the classification labels for snapshot creation are derived from the cost-optimization policy. Consequently, high classification accuracy may partly represent successful reproduction of the underlying decision rule rather than independent discovery of real-world snapshot behaviour. This distinction is important when discussing model effectiveness.

5.6 Threats to Validity and Limitations

The current limitations include:

1. The dataset is based on Bitbrains VM telemetry rather than production Azure snapshot-management logs.
2. Snapshot age is synthetic state rather than a native Bitbrains attribute.
3. Decision labels are generated using the experimental decision policy.
4. Machine-learning performance may therefore be affected by the relationship between input features and generated labels.
5. Real-world snapshot creation and deletion behaviour has not been established as ground truth.
6. Azure deployment and cloud API behaviour may differ from offline experimental execution.
7. The minimum snapshot interval can influence the resulting optimization distribution.
8. Additional validation using real operational snapshot workloads is required.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project presents an AI-powered snapshot lifecycle management system for Azure virtual machines. The system addresses the challenge of balancing snapshot overhead, potential data-loss exposure and recovery requirements.

A cost-based optimization mechanism is used to determine a snapshot interval by considering snapshot overhead and expected data-loss cost. Machine-learning-based decision support using XGBoost is incorporated using workload and recovery-policy features. The system further separates snapshot creation and snapshot retention into distinct lifecycle decisions and incorporates deterministic decision control. An interactive Streamlit dashboard provides visibility into the decision process.

The current evaluation demonstrates the feasibility of the proposed experimental framework. However, the results must be interpreted in the context of synthetic decision labels and telemetry-derived experimental data. Further real-world validation is required before the system can be considered a production-grade autonomous snapshot management solution.

6.1 Future Work

Potential future work includes:

1. Evaluation using real Azure snapshot lifecycle data.
2. Validation using production-scale workloads.
3. Improved modelling of workload volatility.
4. Calibration of machine-learning decision probabilities.
5. More robust sensitivity analysis.
6. Improved handling of uncertainty and model disagreement.
7. Integration with real-time Azure snapshot APIs.
8. Automated policy enforcement after safety validation.

9. Larger and more diverse workload datasets.
10. Improved cost modelling using actual Azure pricing.
11. More comprehensive evaluation of retention policies.
12. Investigation of explainable decision mechanisms.

REFERENCES

- [1] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974. doi:10.1145/361147.361115 [1] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Communications of the ACM*, vol. 17, [1] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974. doi:10.1145/361147.361115 no. 9, pp. 530–531, 1974. doi:10.1145/361147.361115
- [2] J. T. Daly, “A higher order estimate of the optimum checkpoint interval for restart dumps,” *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006. doi:10.1016/j.future.2004.11.016 [2] J. T. Daly, “A higher order estimate of the optimum checkpoint interval for restart dumps,” *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006. [2] J. T. Daly, “A higher order estimate of the optimum checkpoint interval for restart dumps,” *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006. doi:10.1016/j.future.2004.11.016 doi:10.1016/j.future.2004.11.016 [3] A. Benoit et al., “Checkpointing à la Young/Daly: An Overview,” *Int. Conf. on Contemporary Computing (IC3)*, 2022.
- [3] A. Benoit et al., “Checkpointing à la Young/Daly: An Overview,” *Int. Conf. on Contemporary Computing (IC3)*, 2022. [3] A. Benoit et al., “Checkpointing à la Young/Daly: An Overview,” *Int. Conf. on Contemporary Computing (IC3)*, 2022.
- [4] O. Desai, S. Pei, J. Bhimani and B. S. Kim, “Cost-Efficient Training and Checkpointing for Large Models on Pre emptible Cloud VMs,” *EuroMLSys ’26*, pp. 31–40, 2026
- [5] Microsoft Learn, “Back up Azure VMs with the Enhanced policy” and “Guidance and best practices,” *Azure hanced policy* and “Guidance and best practices,” *Azure*
- [6] Microsoft Learn, “Azure Blob Storage lifecycle manage Backup documentation. learn.microsoft.com/en-us/ azure/backup/ Backup documentation. learn.microsoft.com/en-us/ azure/backup/ ment overview.” learn.microsoft.com/en-us/azure/ storage/blobs/lifecycle-management-overview [6] Microsoft Learn, “Azure Blob Storage lifecycle manage
- [7] S. Y. Khan, R. S. A. Larik, A. Bachayo, B. Ali and Kan wal, “ML-Based Adaptive Backup Snapshot Schedulin

PLAGIARISM REPORT

paper.pdf

ORIGINALITY REPORT

2%

SIMILARITY INDEX

2%

INTERNET SOURCES

2%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1

www.ijeat.org

Internet Source

2%

2

Submitted to American National University

Student Paper

<1%

3

www.frontiersin.org

Internet Source

<1%

4

V. Sekhar, P. Ajay Kumar Reddy, K. Logesh, T. Raghunadha Reddy, Dara Raju.
"Next-Generation Smart Systems - Bridging Sustainability and Computational
Intelligence", CRC Press, 2026

Publication

<1%

CONFERENCE/JOURNAL PAPER PUBLICATION

CURRENT SUBMISSION DETAILS

10/6/26, 4:04 PM

Rajalakshmi Engineering College Mail - 28th International Multi-Topic Conference (INMIC 2026) : Submission (237) has been created.



SAI ARAVIND S 230701275 <230701275@rajalakshmi.edu.in>

28th International Multi-Topic Conference (INMIC 2026) : Submission (237) has been created.

Microsoft CMT <noreply@msr-cmt.org>
To: 230701275@rajalakshmi.edu.in

Wed, Sep 30, 2026 at 10:07 PM

Hello,

The following submission has been created.

Track Name: INMIC2026

Paper ID: 237

Paper Title: AI-Powered Snapshot Lifecycle Management and Storage Optimization System

Abstract:

Cloud virtual machines generate continuously changing workloads, making fixed-interval snapshot policies inefficient because they do not account for workload volatility, data-change behavior, recovery requirements, or storage overhead. This paper presents an AI-Powered Snapshot Lifecycle Management and Storage Optimization System for Azure virtual machines that combines analytical cost optimization, workload-risk prediction, and policy-based decision control. The proposed framework analyzes workload telemetry and disk-change behavior to determine an appropriate snapshot creation interval by balancing snapshot overhead against the expected cost of data loss. A separate retention model evaluates snapshot age by balancing storage cost and recovery risk to determine whether retained snapshots should be kept or deleted. In parallel, an XGBoost classifier provides an independent learned decision for snapshot creation using workload and recovery-policy features such as predicted risk, disk-change rate, RPO, RTO, and criticality. The analytical and machine-learning decisions are compared before being passed through a deterministic Safety Policy Engine that enforces minimum snapshot intervals, RPO constraints, retention safeguards, and other operational policies. This layered design separates optimization from execution and prevents model recommendations from directly triggering unsafe or destructive operations. Experiments using workload traces and simulated recovery-policy conditions evaluate snapshot interval selection, retention behavior, decision agreement, and classification performance. The results demonstrate the feasibility of combining cost-aware analytical models, machine learning, and deterministic safeguards for risk-adaptive snapshot lifecycle management in cloud environments.

Created on: Wed, 30 Sep 2026 16:37:40 GMT

Last Modified: Wed, 30 Sep 2026 16:37:40 GMT

Authors:

- 230701275@rajalakshmi.edu.in (Primary)

Primary Subject Area: Artificial Intelligence & Machine Learning

Secondary Subject Areas:

Cloud, Edge & High-Performance Computing

Submission Files:

paper.pdf (515 Kb, Wed, 30 Sep 2026 16:37:28 GMT)

Submission Questions Response:

1. Contact Details
9840035660
2. Format
Yes
3. Originality
Agree, Its not published anywhere
4. Plagiarism
I don't know

<https://mail.google.com/mail/u/0/?ik=8b089d7462&view=pt&search=all&permmsgid=msg-f:1877775578745411268&simpl=msg-f:1877775578745411268> 1/2